COMPUTER ARCHITECTURE 2

1. a) Describe the difference between a write-back and a write-through cache policy.

[2]

Answer:
With a write-through policy, modifications are automatically flushed through to the next level while, with write-back, changes are buffered in the cache until the line is evicted.

(Memorisation. 1 min. to think, 2 mins to write; 3 mins total.)

b) Give two reasons for having separate instruction and data caches in a CPU.

[2]

Answer:

- *Avoids structural hazards in decode and memory pipeline stages.*
- *Instructions and data operate over different areas of memory, so tend to cache different things.*
- *Instructions and data may require different cache parameters to exploit locality.*
- *Split caches can allow location physically closer to the parts of the pipeline that use them, reducing path delays.*

(Bookwork/understanding of locality. 2 mins to think, 2 mins to write; 4 mins total.)

c) Consider a CPU attached to an n-way set-associative cache with 64 bytes/block and 16 blocks. A function executing on the CPU reads data at byte addresses $1024+24i$ for $i \in \{0, 1, 2, \ldots\}$.

i) What is the capacity of the cache in w-bit words?

[2]

Answer:
Capacity in bytes is $2^6 \times 2^4 = 2^{10}$.

Capacity in bits is $2^{10} \times 2^3 = 2^{13}$.

Capacity in words is $2^{13}/w$.

(Standard calculation based on simple formula. 3 mins to read, 1 min. to think; 1 min. to write; 5 mins total.)

ii) What type(s) of locality can be exploited by the cache during a single execution of the function? Only consider the memory accesses of the given data read sequence.

[2]

Answers

Answer:
No piece of data is reread, so there is no temporal locality. However, close pieces of data are used, so there is spatial locality. The reuse occurs within one cache block distance, so the spatial locality is exploitable by this cache.

There may be some temporal locality related to the stack, but that is not relevant here.

We don't care about the associativity of the cache here as there is no temporal locality and the blocks are read sequentially.

(Standard knowledge of spatial/temporal locality. 2 mins to think, 1 min. to write; 3 mins total.)

iii) Assuming that the cache is direct-mapped, estimate the combinatorial delay of a read access. State any extra assumptions that you need to make.

[3]

Answer:
We'll assume that the two-input gate delay is 1 and ignore routing delays. We'll also assume 32-bit words, so tags are each $32-2-4-4=22$ *bits wide.*

There are two paths to calculate: hit/miss and data.

Selection of the appropriate block tag and valid bit requires a level of 16:1 MUXes = 4 gate delays. A level of comparators for the tag follows (1 gate delay), followed by a 23:1 AND reduction tree (5 gate delays). This gives a total of 10 gate delays for the hit/miss path.

Selection of the appropriate data requires a 256:1 MUX = 8 gate delays.

The maximum delay comes from the hit/miss path, so is 10 gate delays.

(Analysis. 4 mins to think/sketch, 3 mins to write; 7 mins total.)

iv) Now assume that the cache is fully associative and starts cold (all blocks invalid). What is the first value of i that will cause an eviction?

[3]

Answer:
The first access is at 1024, which is naturally aligned to a cache line. The offsets are 0, 24, 48, 72, 96, 120, 144, 168, 192 and so on. So, for a sequence of 8 offsets, we encounter 3 cache lines. Given that there are 16 blocks, the first $8 \times {}^{15}/_{3} = 40$ *will use the first 15 lines and the next 3 will use the final line, thus the first eviction will occur for* $i = 43$.

An alternative explanation is that the total capacity is 1024 bytes, and ${}^{1024}/_{24} = 42.667$, *so we can fit up to* $i = floor({}^{1024}/_{24}) = 42$ *before causing overflow.*

(Analysis. 4 mins to think/sketch, 2 mins to write; 6 mins total.)

v) Give an example of high-level C code involving both structs and arrays which might give rise to this address pattern.

Answers

[3]

Answer:
We might have a struct which is 24 bytes in length, declare an array of them and then loop over the first member of each, as in:

```
struct S {
    uint32_t X;
    float Y[5];
};

S a[m];

void f() {
    uint32_t acc = 0;
    for(unsigned i = 0; i < m; i++)
        acc += a[i].X;
}
```

(Synthesis/extrapolation. 3 mins to think, 3 mins to write; 6 mins total.)

vi) Describe how the C code could be optimised or restructured to improve locality, and estimate the resultant performance improvement.

[3]

Answer:
We could move from an array of structures to a structure of arrays, for example:

```
uint32_t a_X[m];
float a_Y[m*5];

void f() {
    uint32_t acc = 0;
    for(unsigned i = 0; i < m; i++)
        acc += a_X[i];
}
```

This would exploit all spatial locality and potentially reduce memory traffic to $4/24 = 1/6^{th}$ *of the original's.*

(Synthesis/extrapolation. 2 mins to think, 2 mins to write; 4 mins total.)

(Question: 38 mins.)

Answers

2. In this question, assume the use of the MIPS-I instruction set architecture.

a) Give two advantages of having a hard-wired zero register.

[3]

Answer:

- *Allows for all-zero encoding of NOP.*
- *Means that unary operations such as SUB or NOT can always be encoded as single binary operations.*
- *Supports common-case pointer operations where there is no offset.*

(Bookwork/coursework. 2 mins to think, 2 mins to write; 4 mins total.)

b) Give a sequence of assembly instructions which evaluates `$r1 += $r2*$r3`, where the three values are all unsigned 32-bit integers and are contained in registers with the same names.

[3]

Answer:

```
MULTU $r2, $r3
MFLO $r2
ADDU $r1, $r1, $r2
```

(Bookwork/coursework. 1 min. to think, 2 mins to write, 3 mins total.)

c) Describe the instruction encoding of `ADDU` and `MULTU`. You should indicate the widths and meanings of the different instruction fields, but do not need to give the bit representations of constants.

[4]

Answer:
Both ADDU and MULTU are R-type, so they have the following fields:

- *opcode: 6 bits. Zero for all R-type instructions.*
- *src1: 5 bits. Identifies one of the input registers for both ADDU and MULTU.*
- *src2: 5 bits. Identifies the other input register.*
- *dst: 5 bits. Identifies the destination register for ADDU, but is not used for MULTU.*
- *shamt: 5 bits. Not used for ADDU or MULTU.*
- *func: 6 bits. Selects the ALU operation, distinguishing between operations including ADDU and MULTU.*

(Bookwork/coursework. 2 mins to think, 3 mins to write/sketch; 5 mins total.)

d) In a particular embedded application, an engineer has found that the CPU spends most of its time calculating `x += y*C`, where `C` is a compile-time constant with at most 6 non-zero bits.

i) Propose a new custom instruction which could be used optimise this type of code, including its instruction encoding.

Answers

[3]

Answer:
We need an instruction which allows us to identify two read registers, a write register and a 6-bit constant. This doesn't fit into traditional R-type, as there is not enough constant space, nor into I-type as it only allows for two registers. However, we can exploit the fact that one of the source registers is shared with the destination register, which does allow for I-type encoding.

The suggested instruction has the pseudo-code:

`MADDC $x, $y, C`

and is encoded as:

- `opcode`: *any spare I-type opcode.*
- `reg1`: `x` *register (both read and write).*
- `reg2`: `y` *register (read only).*
- `imm`: `C` *constant, zero-extended.*

(Analysis/synthesis. 4 mins to think, 2 mins to write; 6 mins total.)

ii) Describe two different approaches for integrating your new instruction into the standard pipelined MIPS datapath, and give the performance advantages of each.

[4]

Answer:
It is difficult to exploit the existing multiplier logic. We could rewrite the instruction at decode to initiate the multiplication, and would then need to insert a stall cycle following the instruction to extract the result and perform an addition. This is very non-RISC, and would seriously impede performance due to the extra logic. Effectively it would be a microcode solution.

A realistic way of implementing it would be to add a specialised block supporting 32×6 *multiplication. This could be:*

- *Inserted into the decode stage, providing* `y*C` *as an input to the ALU in the following stage. This would present a difficult hazard due to the need to potentially forward into* `y`.
- *Merged into the execute stage, increasing the critical path of the ALU. This is the easiest in terms of hazards (no new paths to worry about), but would probably increase the clock period by far too much.*
- *Split across the execute and memory stages. This would have the least impact on clock period, but would require some extra forwarding conditions and introduce possible stalls.*

(Analysis/synthesis. 4 mins to think, 4 mins to write/draw; 8 mins total.)

iii) Pick one approach from the previous answer, and describe any new hazards which could occur and whether each of those hazards should be handled using forwarding and/or stalls.

Answers

[3]

Answer:
Based on these concerns, splitting across the execute and memory stages makes the most sense, i.e. *creating a pipelined unit that performs a fused* `x + y*C`*, independent of and in parallel to the existing ALU and memory operations. There would be two hazard conditions, very similar to those of load instructions, and these could be handled using the same forwarding paths and stalls with conditions added to the forwarding and hazard detection logic.*

There is a write-back to execute stage hazard, handled by stalling and forwarding, and a memory to execute stage hazard, handled by forwarding.

(Follow-on analysis. 3 mins to think, 3 mins to write/draw; 6 mins total.)

(Question: 32 mins)

Answers